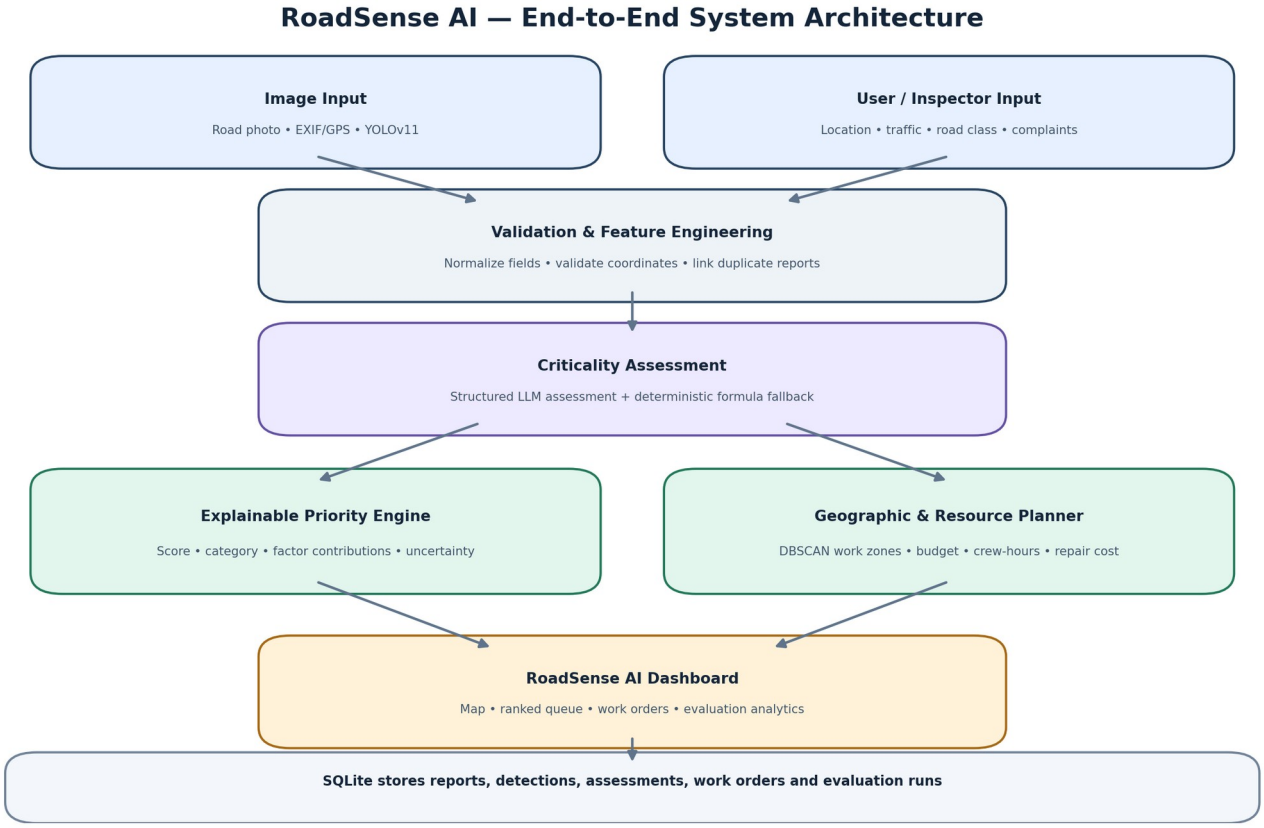
ROADSENSE AI

Smart Road Pothole Prioritization System

Complete Solution, Technology Stack & System Architecture



Hackathon Project Documentation

1. Executive Summary

RoadSense AI is an explainable, resource-aware decision-support system for road authorities. It combines road photographs, YOLOv11 detections, citizen or inspector inputs, geographic context, and maintenance constraints to recommend which damaged locations should be inspected or repaired first. The system does not treat every complaint equally: it assesses damage severity, traffic exposure, road importance, proximity to critical facilities, and urgency, then produces a ranked queue and a feasible maintenance plan.

The hackathon implementation uses a Streamlit web application, Python processing modules, SQLite storage, Folium maps, Plotly charts, scikit-learn geographic clustering, and an optional Anthropic API integration. A deterministic scoring fallback keeps the prototype usable when an AI API is unavailable.

2. Problem Statement

Road authorities receive many reports of potholes and road damage while maintenance resources are limited. Repairing reports in arrival order can delay dangerous defects, and repairing locations independently can waste crew travel and setup time. RoadSense AI supports evidence-based prioritization, budget-aware selection, maintenance planning, and measurable comparison of alternative strategies.

3. Project Objectives

- Detect and represent road damage from images and structured reports.
- Combine multiple risk factors into an understandable priority assessment.
- Respect budget, crew-hours, and daily capacity when selecting repairs.
- Group nearby repair jobs where doing so can reduce mobilization overhead.
- Explain why a location was ranked highly and why a job was deferred.
- Evaluate the proposed strategy against simple baselines under equal resource constraints.

4. System Architecture

The architecture separates perception, feature engineering, criticality assessment, optimization, persistence, and presentation. Each module communicates through validated structured records rather than passing raw model objects or unvalidated AI responses.

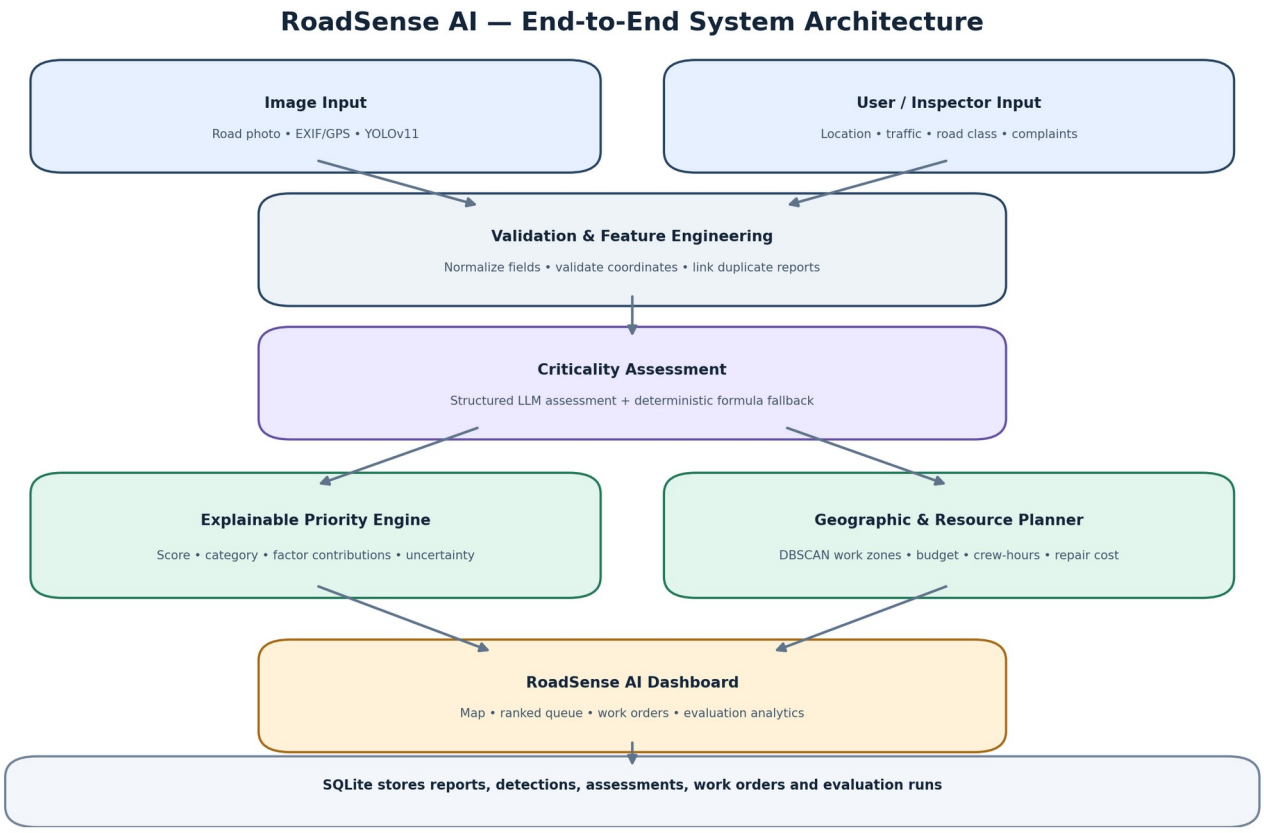


Figure 1. End-to-end architecture of RoadSense AI.

4.1 Processing Flow

1. Input: accept a road image, GPS/map location, road class, traffic level, observed severity, facility proximity, report count, and report age.
2. Detection: run YOLOv11 inference and extract class, confidence, bounding box, and bounding-box area ratio.
3. Validation: validate image and coordinate data, represent missing fields as unknown, and link duplicate complaints to a canonical damage location.
4. Assessment: calculate a transparent formula score and optionally request a structured LLM assessment with a short evidence-based explanation.
5. Planning: cluster nearby locations, estimate costs and repair duration, and select feasible jobs under budget and crew-hour limits.
6. Presentation: show a dashboard, map, ranked queue, work orders, deferred-job reasons, and evaluation charts.
7. Persistence: save reports, detections, assessments, status changes, and evaluation runs in SQLite.

5. Technology Stack and Why It Is Used

Technology	Layer	Reason for use
Python 3.10+	Core language	Integrates computer vision, data processing, AI APIs, optimization, and storage.
Streamlit	Web interface	Rapidly builds interactive forms,

		tables, filters, dashboard metrics, and multipage navigation.
Ultralytics YOLOv11	Image detection	Finds pothole-like objects and returns bounding boxes, classes, and confidence values using trained weights.
Pillow	Image handling	Decodes uploaded images, converts image modes, and supports safe preprocessing.
Anthropic API (optional)	AI assessment	Returns structured criticality assessments and natural-language explanations using a fixed rubric.
pandas + NumPy	Data and features	Combines image features and user inputs, computes scores, and performs vectorized evaluation.
scikit-learn DBSCAN	Geo-clustering	Groups nearby damage locations without requiring a fixed number of clusters.
Folium + streamlit-folium	Interactive map	Displays damage markers, severity colors, and detail popups.
Plotly	Charts	Provides interactive priority distributions, factor contribution charts, and baseline comparisons.
SQLite	Persistence	Stores reports and assessments in a local database without requiring a separate database server.
python-dotenv	Configuration	Loads local environment variables without hardcoding API keys.
Optional OR-Tools	Advanced optimization	Can be added for more rigorous budget, crew assignment, and routing constraints.

6. Input and Data Model

A report is not the same as a pothole. One photo may contain multiple detections, and multiple reports may describe the same damage location. The system should preserve report and detection records separately and create a canonical damage record only after location matching or verification.

Entity	Key fields	Purpose
Report	report_id, submitted_at, image, coordinates, description	Preserves each citizen or inspector submission.
Detection	detection_id, report_id, class, confidence, bbox	Stores each object detected within an image.
Damage location	damage_id, canonical coordinates, road class, severity, status	Represents the repairable road defect.
Assessment	score, category, factor values, reason, model/rubric version	Records how priority was assessed.
Work order	damage_id, crew, schedule, estimated/actual cost, status	Tracks planned and completed maintenance.
Evaluation run	strategy, dataset version, constraints, metrics, seed	Makes strategy comparisons reproducible.

6.1 Important Data Fields

- Coordinates: latitude and longitude from EXIF, a map click, or manual input; validate ranges before use.
- YOLO features: class, confidence, bounding-box coordinates, number of detections, and bounding-box area ratio.
- Context features: road class, traffic level, verified nearby critical facilities, report age, and linked complaint count.
- Maintenance features: estimated repair cost, estimated repair hours, current status, and resource requirements.
- Provenance: indicate whether each value is measured, model-estimated, manually supplied, or unknown.

7. Image Detection and Feature Engineering

The detector loads the configured YOLOv11 weights once and runs inference on each uploaded image. For each bounding box, the application records class, confidence, coordinates, and the ratio of bounding-box pixels to image pixels.

Bounding-box area ratio:

r = (bounding-box width × bounding-box height) / (image width × image height)

This ratio is only a visual proxy. It is not physical pothole area and cannot measure depth. Perspective, camera distance, and viewing angle affect the result. If the model has a validated severity class, it may contribute to severity assessment; otherwise, verified inspection or a clearly marked estimate is needed.

If weights are not configured or inference fails, the application must clearly report that no real detection was performed. Manual reporting and the formula fallback should still work.

8. Explainable Priority Assessment

The fallback scoring engine uses fixed category mappings instead of min-max normalization calculated from each current batch. This prevents a record's score changing merely because unrelated reports were added.

$$P = 0.35S + 0.25T + 0.15R + 0.15C + 0.10U$$

Factor	Weight	Meaning
S — Severity	35%	Observed or verified damage severity; do not infer depth from a 2D box.
T — Traffic	25%	Traffic exposure or an explicitly labeled traffic category.
R — Road importance	15%	Importance of the road class to network movement.
C — Criticality	15%	Verified proximity to hospitals, schools, emergency facilities, or other defined critical locations.

U — Urgency	10%	Capped combination of waiting time and linked complaint history.

S, T, R, C, and U are mapped to 0–100 using fixed documented mappings. P therefore remains between 0 and 100. Initial display bands are Critical ≥ 80, High ≥ 60, Medium ≥ 35, and Low < 35. These are prototype policy settings, not official safety standards.

8.1 AI API Integration

- Send only validated, necessary structured features; do not send reporter names or phone numbers.
- Request one JSON assessment per input ID, including score, category, concise reason, and uncertainty.
- Validate IDs, score range, categories, missing records, duplicates, and output format.
- Cache results using the input feature hash, model ID, and rubric version.
- If the API fails or output is invalid, use deterministic fallback scoring and label the assessment mode.
- Treat AI output as decision support, not as a safety certification. Urgent cases require review under the authority's safety policy.

9. Safety and Data Quality Controls

- Safety override: verified dangerous defects are escalated for urgent review independently of the weighted ranking.
- Missing data: unknown is not the same as zero; display uncertainty and request verification.
- Duplicate reports: link repeated complaints to a canonical damage location rather than creating duplicate repair jobs.
- Coordinate validation: reject invalid latitude/longitude and flag suspicious or missing locations.
- Detection confidence: low-confidence detections should be reviewed rather than silently treated as confirmed defects.
- Privacy and security: validate file type and size, restrict image access, keep secrets out of source control, and define retention rules.

10. Geographic Clustering and Maintenance Planning

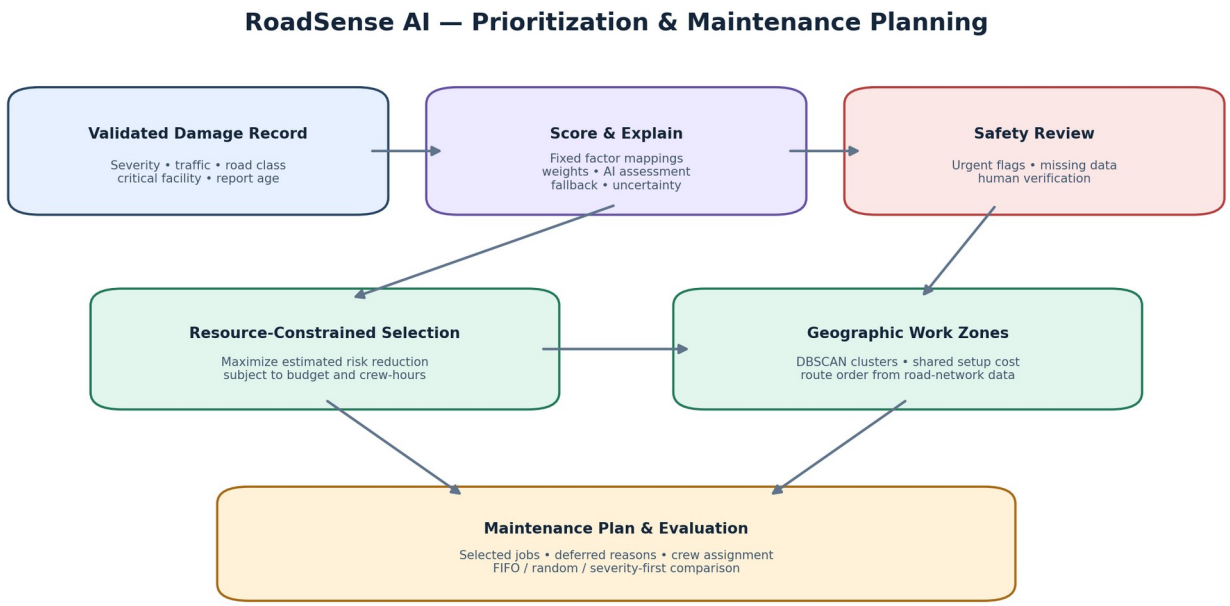


Figure 2. Prioritization and resource-constrained planning logic.

DBSCAN with the haversine distance metric groups damage locations within a configurable geographic radius. A 200-metre radius is a starting configuration, not a universal rule. The DBSCAN noise label (-1) represents isolated locations and must not be treated as a shared cluster.

Shared mobilization cost should be modeled once per work zone or allocated across valid cluster members. A cluster is not a route: route ordering should use road-network travel times when available, because straight-line distances ignore one-way streets, closures, and traffic.

10.1 Resource Selection

Let $x_i = 1$ if repair i is selected and 0 otherwise. A simplified objective is:

$$\textbf{Maximize } \Sigma\, v_i x_i$$

Subject to:

- $\Sigma\, c_i x_i \leq B$ (budget limit)
- $\Sigma\, h_i x_i \leq H$ (available crew-hours, aggregated appropriately across crews)
- Additional operational constraints may include shift duration, equipment, material, safety precedence, and travel time.

Here, v_i is independently defined expected risk reduction, c_i is estimated repair cost, h_i is repair time, B is budget, and H is available crew time. A greedy risk-value-per-cost heuristic is a simple prototype baseline, but it is not guaranteed to find the globally optimal set. OR-Tools can be added for more advanced optimization.

10.2 Required Planner Outputs

- Selected jobs with priority, estimated cost, repair duration, work zone, and proposed crew.
- Deferred jobs with explicit reasons such as budget, crew-hours, already repaired, or lower-ranked candidate.
- Total estimated spend and crew-hours, checked against configured limits.
- Urgent cases that remain unselected, highlighted for review rather than hidden.
- Clear labels distinguishing estimates from actual costs and confirmed schedules.

11. Evaluation Framework

Evaluation should test detection quality, criticality assessment quality, and maintenance strategy quality separately. The system must not claim success solely because it maximizes its own model-generated score.

Evaluation level	Metrics	Evidence needed
Detection	Precision, recall, mAP at stated IoU, false detections/image	Held-out labeled images representative of conditions.
Criticality assessment	Macro-F1, per-class recall, confusion matrix, reviewer agreement, stability	Independent hand-labeled records and separated train/validation/test sets.
Maintenance strategy	Critical-case coverage, independently assessed risk reduction, response time, spend, crew utilization	Same records, budget, and resource constraints for every strategy.

11.1 Baselines

- FIFO: first reported, first repaired.
- Random: seeded random order; repeat across multiple seeds for a fairer estimate.
- Worst-first: highest independently assessed severity first.
- RoadSense AI: multi-factor priority assessment plus resource-aware selection.

Compare strategies at the same budget fractions, for example 10%, 20%, and 30% of total estimated repair cost. Use independent reviewer labels or a documented risk rubric for outcome metrics. If only synthetic or model-generated values are available, report the experiment as a simulation and do not present it as real-world evidence.

12. Application Modules

Page / module	Main features
Overview dashboard	Total reports, critical cases, open work, repaired cases, priority distribution, recent queue.
Report & Detect	Image upload, manual report, structured context, YOLO detections or explicit no-model mode.
Priorities & Map	Filters, ranked table, factor breakdown, map markers, report detail and uncertainty.
Maintenance Planner	Budget and crew-hour controls, work-zone clustering, selected jobs, deferred reasons.
Evaluation	Equal-budget strategy comparison, risk proxy, critical-case

	coverage, saved experiment runs.
Settings	Model configuration status, API status, database location, demo data controls.

13. Database and Code Organization

SQLite is sufficient for a local hackathon prototype. Store structured report and assessment records, work-order status, and evaluation runs. Use parameterized SQL, transactions, stable identifiers, and timestamps. For larger production deployments, consider PostgreSQL/PostGIS.

```
roadsense_ai/
  app.py
  services/
    config.py       # environment configuration
    detection.py    # YOLOv11 inference
    features.py     # validation and feature engineering
    scoring.py      # AI assessment and formula fallback
    geo.py          # coordinate checks and DBSCAN
    planner.py      # budget and crew-hour selection
    evaluate.py     # baselines and metrics
    storage.py      # SQLite persistence
    map_view.py     # Folium map
  tests/
    test_core.py
  requirements.txt
  .env.example
  .gitignore
  README.md
```

14. Implementation Plan

Phase 1 — Foundation

Set up the environment, Streamlit shell, SQLite schema, seeded demo records, and report form.

Phase 2 — Detection and assessment

Integrate trained YOLO weights, validate features, implement fixed scoring, then add optional LLM assessment and fallback.

Phase 3 — Planning

Implement DBSCAN, shared mobilization cost, budget selection, crew-hour constraints, and deferred reasons.

Phase 4 — Dashboard and evaluation

Build map and charts, compare baselines under equal constraints, save evaluation runs, and test the workflow.

Phase 5 — Validation and demo

Test real images, verify outputs, prepare a backup demo, and clearly disclose synthetic data and limitations.

15. Test and Acceptance Checklist

- ☐ Valid image upload either produces actual model detections or explicitly indicates no-model/demo mode.
- ☐ Priority scores remain on a consistent 0–100 scale and show their factor contributions.
- ☐ API errors or invalid JSON trigger deterministic fallback scoring.
- ☐ Duplicate reports can be linked to the same damage location.
- ☐ Invalid coordinates are rejected or flagged before clustering.
- ☐ Noise points do not receive invalid cluster discounts.
- ☐ The planner never exceeds its configured budget or crew-hour capacity.
- ☐ Deferred urgent cases remain visible for safety review.
- ☐ Baseline strategies use the same dataset and resource constraints.
- ☐ Results persist after restarting the local application.

16. Limitations and Production Readiness

- YOLOv11 bounding boxes do not measure pothole depth or physical dimensions.
- Cost and repair-duration estimates must be calibrated with local maintenance data.
- Priority weights and bands are prototype policy settings, not validated public-safety standards.
- An LLM can produce inconsistent or unsupported output; strict validation and human oversight are required.
- DBSCAN groups locations but does not optimize routes by itself.
- The current prototype planner is heuristic and does not guarantee a globally optimal multi-crew schedule.
- Real-world performance claims require independently labeled images, verified road attributes, actual repair outcomes, and field evaluation.

17. Conclusion

RoadSense AI combines computer vision, structured contextual data, explainable prioritization, geographic grouping, resource-aware repair selection, and strategy evaluation in one decision-support workflow. The recommended hackathon approach is to complete the end-to-end workflow with explicit demo data and a reliable fallback first, then integrate and validate the trained YOLOv11 weights and AI API. For operational deployment, add security, authentication, auditability, verified data sources, and a validated maintenance optimization process.